

Using MathCompile

MathCompile is a worksheet of math cells. Type math on the left, and the output column on the right gives you the result in the format you pick: LaTeX source, computed answers, or SymPy Python code. It runs entirely in your browser and saves your worksheet automatically.

The worksheet at a glance

- **Cells** are numbered rows. Click a cell to edit it; the up and down arrow keys move between cells when the caret reaches an edge. **+ Add expression** appends a cell, and **×** deletes one. Pressing Backspace in a cell that holds only blank space deletes it too.
- **The output column** shows whatever the current output target produces for each cell. Drag the divider between the columns to resize it.
- **The header** holds Smart mode, the Output target picker, the dark-mode toggle, and the Commands button (or press `Ctrl+K` / `⌘K`).
- **How do I...**, under the cell list, is a quick keystroke cheat sheet. This document is the long version.

The command palette (`Ctrl+K`) covers: insert an expression below, duplicate the current one, delete it, clear all expressions, toggle Smart mode and dark mode, switch the output target, and jump to any cell by number.

Typing math

Smart mode (the default)

With Smart mode on — the checkbox in the header — you rarely need a backslash. Whole words convert to symbols as you type, and a digit right after a letter becomes a subscript:

Type	Get
int	$\int$ with upper and lower bound blocks
iint or antid	a bare $\int$ for indefinite integrals
sum , prod	Σ , Π with bound blocks
sqrt	$\sqrt{}$
pi , theta , infty	π , θ , ∞
derivative	$\frac{d}{d}$ — caret lands in the denominator
def	the function-definition marker (see <i>Output</i>)
x2	x_2

Operator names like `sin` , `lim` , and `det` stay upright and space correctly without any command.

LaTeX commands

Type `\` followed by a command name — `\frac`, `\sqrt`, `\int`, `\alpha` — and press **Enter** or **Space** to accept it. As you type, a suggestion menu lists matching commands; the arrow keys highlight and Enter accepts the highlight, or just keep typing to filter. `Esc` dismisses the menu. Word-runs of two or more letters suggest too (`sq` offers `\sqrt`).

`Ctrl+Space` opens the searchable symbol picker: type to filter, Enter inserts, `Esc` closes.

Unicode characters insert directly — typing `α`, `≤`, `∫`, `∀` just works.

Common things you'll type

You want	Type
Fraction	<code>1/2</code> , or <code>\frac</code> then fill the blocks
Subscript / superscript	<code>_</code> , <code>^</code> (or <code>x2</code> in Smart mode)
Square / nth root	<code>sqrt</code> , or <code>\nthroot</code>
Sum / product with bounds	<code>sum</code> or <code>prod</code> , then <code>_</code> and <code>^</code>
Definite integral	<code>int</code> , then <code>_</code> and <code>^</code>
Indefinite integral	<code>iint</code> or <code>antid</code> — add bounds later with <code>_</code>
Limit	<code>\lim</code> , then <code>_</code> for $x \rightarrow 0$
Binomial coefficient	<code>\binom</code>
Matrix	<code>\pmatrix</code> — Enter adds a row, <code>Shift+Space</code> a column
Piecewise	<code>\cases</code>
Greek letters	<code>\alpha</code> , <code>\beta</code> , ... or <code>\pi</code> , <code>\theta</code> , <code>\infty</code>
Text inside math	<code>\text{...}</code>
Accents	<code>\dot</code> , <code>\ddot</code> , <code>\bar</code> , <code>\hat</code> , <code>\vec</code> , <code>\tilde</code>
Sets	<code>\in</code> , <code>\notin</code> , <code>\subseteq</code> , <code>\cup</code> , <code>\cap</code> , <code>\emptyset</code> , <code>\mathbb{R}</code>
Relations	<code>\leq</code> , <code>\geq</code> , <code>\neq</code> , <code>\approx</code> , <code>\pm</code>
Arrows	<code>\to</code> , <code>\rightarrow</code> , <code>\Leftrightarrow</code>
Calculus	<code>\partial</code> , <code>\nabla</code> , <code>\infty</code> , <code>\prime</code>
Logic	<code>\forall</code> , <code>\exists</code> , <code>\therefore</code> , <code>\land</code> , <code>\lor</code> , <code>\not</code>

`SYMBOLS.md` in the repository lists every command the editor accepts.

Getting in and out of blocks

Commands like `\frac` open placeholder blocks, and the caret stays inside the block you're filling. Everything you type lands there — including `+` , `=` , and spaces. To move on:

- **Right** or **Tab** steps out of the current block.
- `(` `[` `{` insert closed pairs with the caret inside; typing the closer again hops over it. `\angle` pairs with `\rangle` .

- Backspace clears the block you're in before it removes the command that owns it.

Multi-line cells and cells

Enter starts a new line inside the same cell (useful for aligned derivations); **Shift+Enter** starts a whole new cell. While a `\command` input is still open, Enter accepts the command instead of breaking a line.

Linear algebra

Environments open two ways: type `\name` directly (the shortcut) or `\begin{name}` the long way. In the table, each entry shows the shortcut and what a `a,b / c,d` grid renders as.

Matrices

Type	Renders
<code>\pmatrix</code>	$\begin{pmatrix} a & b \\ c & d \end{pmatrix}$
<code>\bmatrix</code>	$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$
<code>\Bmatrix</code>	$\begin{Bmatrix} a & b \\ c & d \end{Bmatrix}$
<code>\vmatrix</code>	$\begin{vmatrix} a & b \\ c & d \end{vmatrix}$
<code>\Vmatrix</code>	$\begin{Vmatrix} a & b \\ c & d \end{Vmatrix}$
<code>\matrix</code>	$\begin{matrix} a & b \\ c & d \end{matrix}$
<code>\smallmatrix</code>	$\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}$

Inside a matrix grid: **Enter** adds a row, **Shift+Space** adds a column, and the arrow keys walk cell to cell. `&` separates columns and `\\` ends a row in the stored LaTeX. The bare forms `\pmatrix{a&b\\c&d}` and friends also work — one brace argument, no `\begin`.

Matrix operations

`det` is an upright name (works with or without Smart mode); the rest of the ops go through `\mathrm{name}` — the compiler reads upright names as functions, so these become real matrix methods in the Calculator and Python targets. A name that isn't built in, like `\rank`, types as plain `\text{rank}` — not an operator.

Operation	Type	Renders
Determinant	<code>det</code> then the matrix	det A
Trace	<code>\mathrm{trace}</code> (or <code>\mathrm{tr}</code>)	trace (A)
Rank	<code>\mathrm{rank}</code>	rank (A)

Operation	Type	Renders
Inverse	<code>A^{-1}</code> or <code>\mathrm{inverse}</code>	A^{-1}
Transpose	<code>A^{\mathrm{T}}</code> or <code>\mathrm{transpose}</code>	A^T
Norm	<code>\lVert</code> (auto-pairs)	$\ A\ $
Eigenvalues	<code>\mathrm{eigenvals}</code>	$\mathrm{eigenvals}(A)$
Eigenvectors	<code>\mathrm{eigenvects}</code>	$\mathrm{eigenvects}(A)$
Kernel / null space	<code>\ker</code>	$\ker A$

Multi-line equations

Type	Renders
<code>\gathered</code>	$ \begin{array}{l} x + y = 1 \\ x - y = 3 \end{array} $
<code>\aligned</code>	$ \begin{array}{l} x = y \\ a = b \end{array} $
<code>\cases</code>	$ \begin{cases} x^2 & x > 0 \\ 0 & x \leq 0 \end{cases} $

Aliases open the same grid: `\gather`, `\equation`, `\multline`, and `\flalign` all open a centered stack like `\gathered`; `\align` and `\split` open an aligned grid. `\begin{...}` additionally accepts `eqnarray` and `eqalign` (also aligned grids) — those two have no `\name` shortcut. Rows and cells edit like matrices: Enter adds a row, & separates columns.

Custom grids

Grids that take a column spec or a count open a pending `{ }` block when you type the `\name` — enter the argument (`cc`, `l|r`, `2`) and press `}`, Enter, or Tab to apply it.

Type	Renders
<code>\array{lr}</code>	$ \begin{array}{lr} a & b \\ cc & d \end{array} $
<code>\tabular{ll}</code>	a spec'd grid like <code>\array</code> , for text content
<code>\subarray{c}</code>	$ \begin{subarray}{c} a \\ b \end{subarray} $
<code>\alignat{2}</code> / <code>\alignedat{2}</code>	$ \begin{array}{ll} x = y & u = v \\ a = b & c = d \end{array} $
<code>\substack</code>	$ \begin{substack} a \\ b \end{substack} $

Set theory

The named sets are single-letter commands — `\R`, `\N`, `\Z`, `\Q`, `\C`, `\P`, `\H`. Spelling out `\mathbb{R}` doesn't resolve as a command; use the letter.

Type	Renders
<code>\R</code> (same pattern for <code>\N</code> <code>\Z</code> <code>\Q</code> <code>\C</code> <code>\P</code> <code>\H</code>)	$\mathbb{R}$
<code>\Z</code> then <code>_n</code>	$\mathbb{Z}_n$
<code>x \in A</code> , <code>\notin</code>	$x \in A, x \notin A$
<code>\cup</code> <code>\cap</code> <code>\setminus</code>	$A \cup B, A \cap B, A \setminus B$
<code>\subset</code> <code>\subseteq</code> <code>\nsubseteq</code>	$A \subset B, A \subseteq B, A \not\subseteq B$
<code>\emptyset</code> or <code>\varnothing</code>	$\emptyset$
<code>\{1,2,3\}</code>	$\{1,2,3\}$ — braces auto-pair
<code>\{ x \in \mathbb{R} \mid x > 0 \}</code>	$\{x \in \mathbb{R} \mid x > 0\}$
<code>\bigcup</code> <code>\bigcap</code> , then <code>_{i=1}^n</code>	$\bigcup_{i=1}^n A_i, \bigcap_{i=1}^n A_i$
<code>(a,b)</code> , <code>[a,b)</code> , <code>[a,b]</code>	$(a,b), [a,b), [a,b]$ — parens/brackets auto-pair

These compile to real SymPy sets: `x \in \mathbb{R}` puts `real=True` on `x`'s symbol (likewise `\Z` → integer, `\Q` → rational, `\C` → complex), `\cup` / `\cap` / `\setminus` become `Union` / `Intersection` / `Complement`, `\{1,2,3\}` a `FiniteSet`, `\{x \mid p(x)\}` a `ConditionSet`, and `\{f(x) \mid x \in D\}` an `ImageSet`. A `\subseteq B` needs concrete sets on both sides — SymPy can't name an unknown set, so bare symbols get an `!` error note.

Logic

Type	Renders
<code>\land</code> <code>\lor</code> <code>\neg</code>	$p \wedge q, p \vee q, \neg p$
<code>\implies</code> <code>\iff</code>	$p \implies q, p \iff q$
<code>\forall</code> <code>\exists</code>	$\forall x \in S, \exists x \in S$
<code>\vdash</code> <code>\models</code>	$T \vdash p, M \models p$
<code>\top</code> <code>\bot</code>	$\top, \perp$
<code>\therefore</code> <code>\because</code>	$\therefore, \because$

Connectives lower to `And` / `Or` / `Not` / `Implies` / `Equivalent`. A quantifier over a concrete set folds elementwise (`\forall x \in \{1,2,3\}` becomes an `And` over the elements); SymPy has no general quantifiers, so an abstract `\forall` is flagged rather than computed.

Calculus

`\int`, `\sum`, and `\prod` come with `_ / ^` bound blocks already made — type the bound, then `Right` or `Tab` to the next block. A typed `_ / ^` inside a bound nests a block instead of moving. `\lim` has no pre-made bound: type `_{x \to 0}` yourself.

Type	Renders
<code>\int</code> , bounds <code>0</code> then <code>1</code> , <code>x^2</code> <code>Right</code> <code>dx</code>	$\int_0^1 x^2 dx$

Type	Renders
<code>\antid</code>	$\int f \, dx$ — indefinite (adds $+C$ in the calculator)
<code>\iint</code> <code>\iiint</code>	$\iint, \iiint$ — boundless signs
<code>\oint</code> <code>\oiint</code>	$\oint, \oiint$ — closed integrals
<code>\frac{d}{dx}</code> or <code>\derivative</code>	$\frac{dy}{dx}$
<code>f'</code> , <code>f''</code> , <code>f^{(n)}</code>	$f'(x), f''(x), f^{(n)}(x)$
<code>\sum</code> then <code>k=1</code> Right <code>n</code> ; <code>\prod</code> same	$\sum_{k=1}^n a_k, \prod_{k=1}^n a_k$
<code>\lim</code> then <code>_ {x \to 0}</code> ; one-sided <code>\theta^{+}</code>	$\lim_{x \rightarrow 0} f(x), \lim_{x \rightarrow 0^+} f(x)$
<code>\lim</code> , <code>\limsup</code> , <code>\liminf</code>	<code>\lim</code> , <code>\lim sup</code> , <code>\lim inf</code> — upright operator names
<code>\infty</code>	∞

Integrals, derivatives, sums, products, and limits all compile to the real SymPy calls — `integrate`, `diff`, `Sum / Product`, `limit` (one-sided included).

Multivariable calculus

Type	Renders
<code>\partial</code>	∂
<code>\frac{\partial u}{\partial t}</code>	$\frac{\partial u}{\partial t}$
<code>\partial_x</code> then <code>u</code>	$\partial_x u$ — reads as the same partial derivative
<code>\iint</code> , <code>\iiint</code> , then <code>f</code> <code>dx dy</code>	$\iint f \, dx \, dy, \iiint f \, dx \, dy \, dz$
<code>\oiint</code>	$\oiint$ — closed surface integral
<code>\nabla</code>	$\nabla f, \nabla^2 u$
<code>\Delta</code>	Δu
<code>\vec{v}</code>	$\vec{v}$

`\partial_x u` compiles to the partial derivative like the `\frac` form. `\nabla` and `\Delta` are display notation — the compiler reads `\nabla` as an ordinary symbol, so `\nabla f` stays symbolic rather than becoming a gradient call.

Topology

Type	Renders
<code>f : X \to Y</code>	$f : X \rightarrow Y$ — a continuous map
<code>\mapsto</code>	$x \mapsto x^2$
<code>\cong</code>	$X \cong Y$ — homeomorphic
<code>\times</code>	$X \times Y$ — product
<code>\bar{A}</code> or <code>\overline{A}</code>	$\overline{A}$ — closure

Type	Renders
<code>\partial A</code>	∂A — boundary
<code>A^{\circ}</code>	A° — interior
<code>A'</code>	A' — complement
<code>(a,b)</code> , <code>[a,b)</code>	(a,b) , $[a,b)$ — open/half-open sets
<code>\varepsilon</code> <code>\delta</code>	ε, δ

Topology rows are notation — unions, intersections, closures, and maps display but carry no topological semantics. One exception to know: `\cong` is read by the compiler as modular congruence ($x \cong b \pmod m \rightarrow \text{Eq}(\text{Mod}(x, m), b)$), not isomorphism.

Complex analysis

Type	Renders
<code>\C</code>	$\mathbb{C}$
<code>\Re</code> <code>\Im</code>	$\Re z, \Im z$
<code>\arg</code>	$\arg z$
<code>\bar{z}</code> or <code>\overline{z}</code>	$\bar{z}$ — conjugate
<code> z </code>	$ z $ — modulus; <code> </code> auto-pairs
<code>e^{i\theta}</code>	$e^{i\theta}$
<code>\oint</code>	$\oint_{\gamma} f(z) dz$ — contour integral
<code>\wp</code> <code>\ell</code>	$\wp, \ell$

`\Re`, `\Im`, `\arg`, and `\overline{z}` compile to `re`, `im`, `arg`, and `conjugate`; `i` is the imaginary unit.

Abstract algebra

Type	Renders
<code>\Z</code> then <code>_n</code>	$\mathbb{Z}_n$ — integers mod n
<code>\langle a \rangle</code>	$\langle a \rangle$ — generated subgroup; auto-pairs
<code>\times</code> <code>\oplus</code> <code>\otimes</code>	$G \times H, G \oplus H, G \otimes H$
<code>\circ</code>	$f \circ g$ — composition
<code>\star</code>	$a \star b$ — generic operation
<code>\ker</code> <code>\phi</code>	$\ker \phi$ — kernel of a map
<code> G </code>	$ G $ — order
<code>\trianglelefteq</code>	$N \trianglelefteq G$ — normal subgroup
<code>G/H</code>	G/H — quotient
<code>\mathfrak{g}</code>	$\mathfrak{g}$ — Lie algebra

Type	Renders
<code>x \equiv b \pmod{m}</code>	$x \equiv b \pmod{m}$

The modular form compiles: `x \equiv b \pmod{m}` lowers to `Eq(Mod(x, m), b)`. Group notation (`\langle`, `\circ`, `\star`, `\trianglelefteq`) is display-level — infix symbols, no algebraic structure attached.

Differential equations

Type	Renders
<code>y'</code> , <code>y''</code>	y', y''
<code>\dot{x}</code> <code>\ddot{x}</code>	$\dot{x}, \ddot{x}$ — time derivatives
<code>f^{(n)}(x)</code>	$f^{(n)}(x)$ — nth derivative
<code>\frac{dy}{dx}</code>	$\frac{dy}{dx}$
<code>\text{def} y(t)</code>	declares <code>y</code> a function of <code>t</code>
<code>\mathcal{L}</code>	$\mathcal{L}\{f\}$ — Laplace transform notation

Declaring `\text{def} y(t)` marks `y` as a dependent variable, so `y'`, `y''`, and `y^{(n)}` emit real `Derivative / diff` calls in `t`. Dotted names read as functions of `t` on their own — `\dot{x}` is `x(t)` differentiated once. `\mathcal{L}` is notation only.

Partial differential equations

Type	Renders
<code>\frac{\partial u}{\partial t}</code>	$\frac{\partial u}{\partial t}$
<code>\partial_t</code> then <code>u</code>	$\partial_t u$ — same operator, shorter
<code>\nabla^2</code> <code>u</code> or <code>\Delta</code> <code>u</code>	$\nabla^2 u, \Delta u$ — Laplacian
<code>\nabla \cdot</code> / <code>\nabla \times</code>	$\nabla \cdot \vec{F}, \nabla \times \vec{F}$
<code>u_t</code> , <code>u_{xx}</code>	u_t, u_{xx} — subscript notation
<code>\infty</code>	∞ — conditions at infinity

`\partial_t u` compiles to the partial derivative, like the `\frac` form. The `\nabla`, `\Delta`, divergence, and curl spellings are display notation — no vector-calculus semantics behind them.

Combinatorics

Type	Renders
<code>\binom{n}{k}</code>	$\binom{n}{k}$ — <code>binomial(n, k)</code>
<code>n!</code> , <code>n!!</code>	$n!, n!!$ — factorial / double factorial
<code>\lfloor x \rfloor</code> , <code>\lceil x \rceil</code>	$\lfloor x \rfloor, \lceil x \rceil$
<code>\sum</code> / <code>\prod</code> with bounds	$\sum_{k=0}^n, \prod_{k=1}^n$

Type	Renders
<code>\Gamma(z)</code>	$\Gamma(z)$ — <code>gamma(z)</code>
<code>\mathrm{totient}(n)</code>	totient (<i>n</i>) — Euler's totient
<code>\dots</code> <code>\cdots</code>	1, 2, ..., n

`\binom`, `! / !!`, floors, ceilings, and `\Gamma` compile to the matching SymPy calls. Other number-theory words work like `totient` via `\mathrm{name}` — `mobius`, `nextprime`, `factorint` — as long as the name is a plain word.

Output targets

Pick a target from the **Output** menu in the header.

LaTeX

The output column shows each cell's serialized LaTeX; **Copy** puts it on the clipboard.

Calculator

Each cell evaluates through SymPy — a real computer-algebra system that runs in the page as WebAssembly. It takes a while to download and boot; until the engine is ready, results come from a lightweight JavaScript engine and are dimmed and marked *estimate*. The chip at the top shows engine status.

- Each statement in a cell gets a result row; `≈` adds a decimal approximation when one exists.
- `!` marks a statement that failed, `i` a note; hover for the message.
- **Show generating code** reveals the exact Python program that produced the results, with a copy button.
- The **Settings** menu (calculator and Python targets) exposes display plumbing and animation timing — default values are fine.

Python

The output column shows the SymPy program MathCompile generates per cell, syntax-highlighted. **Copy** grabs one cell's lines; **Copy script** grabs the whole worksheet as one runnable program. The `import *` checkbox controls the import style. Statements the compiler can't express appear as `! / i` notes under the code.

One rule to know: write `\text{def} f(x) = 2x` (type `def` in Smart mode) to define a function — the generated code is `def f(x): return 2*x`. A bare `f(x)` is read as multiplication, `f * x`.

Keyboard reference

Key	What it does
<code>Enter</code>	new line in the cell; accepts an open <code>\command</code> ; adds a row in a matrix
<code>Shift+Enter</code>	new cell below
<code>Right</code> , <code>Tab</code>	step out of the current block

Key	What it does
Esc	dismiss the autocomplete menu or picker; returns focus to the field
Ctrl+Space	open the symbol picker
Ctrl+K / ⌘K	command palette
Shift+Space	new column inside a matrix
↑ , ↓	move between cells at field edges
Home , End	block edges (Ctrl+Home / Ctrl+End for the whole field)
Backspace on a blank cell	delete the cell

Saving

The worksheet, theme, Smart mode, and target choice persist in your browser's local storage — there's nothing to click. Clearing the site's data resets everything.

Repository docs: [README.md](#) (build/run), [SYMBOLS.md](#) (command inventory), [AGENTS.md](#) (architecture), [CALCULATOR-ENGINES.md](#) (calculator internals).